

Rational Matrix Policy Optimization for Transformer Language Models

Anonymous authors
Paper under double-blind review

Abstract

1 Introduction

2 Background and Related Work

3 Rational Matrix Policy Optimization

Rational Latent Basis (RLB) parameterizes each Transformer MLP sublayer as grouped rational features with explicit input and output matrix factors. The term latent refers to the learned projected coordinates z_l , not to local atom centers. This exposes groupwise sensitivities, relative update pressures, and a positive scale gauge that a standard optimizer does not distinguish. MatrixPolicy is the corresponding matrix-specific optimizer rule. Our main configuration uses the fused global-rational RLB variant: a single-branch RLB sublayer with grouped trainable degree-5 numerator and degree-4 denominator rational curves, no local atom branch, and no atom parameter group.

3.1 Global-Rational RLB Sublayer

For a Transformer layer l , let $x_l \in \mathbb{R}^d$ be the MLP input. RLB replaces the usual gated/value activation by a single rational branch

$$z_l = A_l x_l, \quad z_{l,g} \in \mathbb{R}^m, \quad H = Gm, \quad (1)$$

where $A_l \in \mathbb{R}^{H \times d}$ and the hidden coordinates are partitioned into G groups. Each group is evaluated in normalized coordinates,

$$r_{l,g} = \sqrt{m^{-1} \|z_{l,g}\|_2^2 + \epsilon_{\text{rms}}}, \quad u_{l,g} = z_{l,g} / r_{l,g}, \quad (2)$$

then mapped coordinatewise by a learned rational curve and rescaled by the same radius:

$$h_{l,g} = r_{l,g} R_{l,g}(u_{l,g}), \quad y_l = B_l \text{concat}_{g=1}^G h_{l,g}. \quad (3)$$

Here $B_l \in \mathbb{R}^{d \times H}$ recombines the rational features. The global-rational curve is

$$R_{l,g}(u) = \frac{P_{l,g}(u)}{Q_{l,g}(u)}, \quad P_{l,g}(u) = \sum_{i=0}^5 a_{l,g,i} u^i, \quad (4)$$

with the pole-safe denominator

$$Q_{l,g}(u) = 1 + |b_{l,g,1}| |u| + |b_{l,g,2}| u^2 + |b_{l,g,3}| |u|^3 + |b_{l,g,4}| u^4. \quad (5)$$

All denominator terms are nonnegative on the real line, so $Q_{l,g}(u) \geq 1$ and there are no real poles. The coefficients are initialized from a SiLU fit on a fixed probe interval and then trained. Because equation 5 contains absolute-value terms, the curve is pole-safe and piecewise smooth rather than a globally smooth rational polynomial quotient. Global here means that the nonlinear correction is the groupwise rational curve itself, with no local atom basis around fixed centers; coefficients remain layer- and group-specific.

3.2 Why This Parameterization Matters

RLB separates three roles that a standard optimizer would otherwise treat nearly uniformly. The input matrix A_l chooses the rational domains; the rational coefficients set the response and derivative profile in those domains; the output matrix B_l recombines rational features into the residual stream. Write $f_l(x; A_l, B_l, R_l)$ for the RLB block map defined above, where R_l collects all group rational curves in layer l . The group RMS normalization also creates a positive scale gauge between A_l and B_l . In the homogeneous idealization $\epsilon_{\text{rms}} = 0$, for nonzero group radii, any positive group scales $a_g > 0$, and block matrix $D_l(a) = \text{diag}(a_1 I_m, \dots, a_G I_m)$,

$$f_l(x; A_l, B_l, R_l) = f_l(x; D_l(a)A_l, B_l D_l(a)^{-1}, R_l). \quad (6)$$

Thus many matrix representatives encode the same block function. The training rule uses a stabilizing RMS floor, so equation 6 is not an exact training-time invariance. We use it as a design principle: the optimizer should control motion in the represented function while preventing uncontrolled drift in the matrix representative. Appendix A.2 proves the homogeneous identity and states the floor-induced discrepancy.

In the main configuration, MatrixPolicy is applied only to the RLB matrices $\{A_l, B_l\}_{l=1}^L$. All non-matrix parameters, including attention, embeddings, layer norms, and the rational numerator/denominator coefficients, are updated by the AdamW-backed non-matrix optimizer. This partition is important: the method does not rely on a separate coefficient optimizer, and it does not introduce any local-atom coefficient group.

3.3 MatrixPolicy

MatrixPolicy combines three operations: on-policy group scaling, a role/depth matrix update, and a post-step gauge rebalance. Let $r \in \{\text{in}, \text{out}\}$ denote the matrix role, with $M_{l,\text{in}} = A_l$ and $M_{l,\text{out}} = B_l$. We write $A_{l,g} \in \mathbb{R}^{m \times d}$ for the input row block of group g and $B_{l,g} \in \mathbb{R}^{d \times m}$ for the matching output column block. Before the optimizer step, MatrixPolicy records relative update pressures for each rational group,

$$p_{\text{in},g} = \frac{\text{rms}(\nabla A_{l,g})}{\text{rms}(A_{l,g})}, \quad p_{\text{out},g} = \frac{\text{rms}(\nabla B_{l,g})}{\text{rms}(B_{l,g})}, \quad p_{\text{rat},g} = \text{rms}(\nabla \theta_{R,l,g}), \quad (7)$$

where $\theta_{R,l,g}$ denotes the raw numerator coefficients and denominator parameters before the absolute-value map in equation 5. Let $q_{\text{in},g}$, $q_{\text{out},g}$, and $q_{\text{rat},g}$ be EMAs of these quantities, with all RMS values and EMAs floored by a small positive constant before logarithms. MatrixPolicy uses

$$\pi_g = \log q_{\text{in},g} - \log q_{\text{out},g}, \quad \alpha_g = \log q_{\text{rat},g} - \frac{1}{2}(\log q_{\text{in},g} + \log q_{\text{out},g}), \quad (8)$$

as pressure and rational-activity signals. Let T be the training-step budget, $\xi_t = t/T$, and let $\sigma_{a,b}(\xi)$ be the smoothstep gate from 0 to 1 over $[a, b]$. We write $\tilde{\pi}_g = \text{clip}(\pi_g, -p_{\text{max}}^{\text{grp}}, p_{\text{max}}^{\text{grp}})$ and $\phi_t = \sigma_{0.02, 0.30}(\xi_t)$ for the clipped pressure and group-policy schedule. The geometric mean over groups is $\text{geomean}_g(c) = \exp(G^{-1} \sum_{g=1}^G \log c_g)$.

The group-gradient multiplier is centered and clipped:

$$\begin{aligned} c_{g,r} &= c_{g,r}^{\text{gain}} c_{g,r}^{\text{pressure}} c_g^{\text{activity}}, \\ c_{g,r} &\leftarrow c_{g,r} / \text{geomean}_g(c_{g,r}), \\ c_{g,r} &\leftarrow \text{clip}(c_{g,r}, c_{\text{min}}, c_{\text{max}}). \end{aligned} \quad (9)$$

For the input matrix, the gain term uses the rational derivative RMS d_g ; for the output matrix, it uses the rational output RMS o_g measured on normalized coordinates,

$$d_g = \text{rms}(R'_{l,g}(u_{l,g})), \quad o_g = \text{rms}(R_{l,g}(u_{l,g})), \quad (10)$$

with the RMS taken over sampled normalized coordinates and stabilized by the same small positive floors used in training. The gain factors are

$$c_{g,\text{in}}^{\text{gain}} = \left(\frac{\text{geomean}(d)}{d_g} \right)^{\lambda_g \phi_t}, \quad c_{g,\text{out}}^{\text{gain}} = \left(\frac{\text{geomean}(o)}{o_g} \right)^{\lambda_g \phi_t}. \quad (11)$$

The pressure term moves the two matrix roles in opposite directions,

$$c_{g,\text{in}}^{\text{pressure}} = \exp(-\lambda_p \phi_t \tilde{\pi}_g), \quad c_{g,\text{out}}^{\text{pressure}} = \exp(+\lambda_p \phi_t \tilde{\pi}_g), \quad (12)$$

and the rational-activity term damps matrix updates when coefficient gradients are large relative to matrix gradients. Here α_0 is the activity target and $w_\alpha > 0$ is the activity transition width:

$$c_g^{\text{activity}} = \exp \left[-\lambda_a \phi_t \text{ReLU} \left(\frac{\alpha_g - \alpha_0}{w_\alpha} \right) \right]. \quad (13)$$

Geometric centering keeps the average log multiplier at zero before the final clip; the clip bounds the influence of noisy per-group statistics. In the main configuration, $(\lambda_g, \lambda_p, \lambda_a) = (0.20, 0.10, 0.20)$, $(\alpha_0, w_\alpha) = (0.05, 0.45)$, $p_{\text{max}}^{\text{grp}} = 1.50$, and $(c_{\text{min}}, c_{\text{max}}) = (0.75, 1.35)$.

After group scaling, each RLB matrix receives a role/depth AdamW step and, when active, a matrix-normalized step. The AdamW matrix scale is

$$a_{l,r} = \text{clip}\{3.0 a_{\text{role}}(l, r), 0.40, 4.0\}. \quad (14)$$

The role factor depends on normalized depth $d_l = (l - 1)/(L - 1)$:

$$\rho_{\text{in}}(l) = \text{clip}(1 - 0.50(d_l - 0.5), 0.55, 1.40), \quad \rho_{\text{out}}(l) = \text{clip}(1 + 1.00(d_l - 0.5), 0.55, 1.40), \quad (15)$$

with $a_{\text{role}}(l, r) = \max(0.10, 1 + 1.20(\rho_r(l) - 1))$. The matrix-normalized branch is active only early. Let η_t be the shared base learning-rate schedule and define

$$\mu_{l,r,t} = \text{clip} \left(0.75 \sigma_{0.02, 0.12}(\xi_t) [1 - \sigma_{0.20, 0.36}(\xi_t)] \rho_r(l) s_{\text{stat}}(l, r, t), 0, 0.75 \right), \quad (16)$$

where $s_{\text{stat}}(l, r, t)$ is a bounded pressure/activity factor defined in Appendix B. Operationally, MatrixPolicy composes two stateful optimizer steps with temporary learning-rate multipliers:

$$M'_{l,r} = \text{AdamW}_{\eta_t a_{l,r}(1-\mu_{l,r,t})}(M_{l,r}^t; \nabla M_{l,r}^t), \quad (17)$$

$$M_{l,r}^{t+1} = \text{MatNorm}_{\eta_t \mu_{l,r,t}}(M'_{l,r}; \nabla M_{l,r}^t). \quad (18)$$

Here MatNorm denotes the early Muon-style matrix-normalized branch used for RLB matrices: it maintains its own matrix-gradient momentum and normalizes the resulting matrix update direction before applying the temporary learning-rate multiplier. This notation describes the step composition and learning-rate multipliers, not a claim that the two stateful updates are an exact additive closed form. After $\mu_{l,r,t}$ permanently decays to zero, the zero-learning-rate matrix-normalized branch is omitted. This reduces runtime without changing the nonzero updates.

The final operation is a clipped positive-gauge rebalance. Define the current log-ratio

$$\gamma_g = \log \text{rms}(A_{l,g}) - \log \text{rms}(B_{l,g}). \quad (19)$$

Using derivative gain d_g , output gain o_g , and the separately clipped gauge pressure $\bar{\pi}_g = \text{clip}(\pi_g, -p_{\text{max}}^{\text{bal}}, p_{\text{max}}^{\text{bal}})$, MatrixPolicy defines the gauge target

$$\tau_g = w_\tau \frac{1}{2} (\log d_g - \log o_g) + \beta_p \bar{\pi}_g, \quad (20)$$

where w_τ weights the gain-ratio target and equals 1 in the main configuration. Its activity factor is

$$a_g^{\text{activity}} = \exp\{\text{clip}(\beta_a \bar{\alpha}_g, \log a_{\text{min}}, \log a_{\text{max}})\}, \quad (21)$$

where $\bar{\alpha}_g = \text{clip}(\alpha_g, -p_{\text{max}}^{\text{bal}}, p_{\text{max}}^{\text{bal}})$, β_a is the activity-gain coefficient, and $0 < a_{\text{min}} \leq a_{\text{max}}$. Activity can either damp or amplify the scheduled gauge move within this fixed range. MatrixPolicy applies

$$\ell_g = \text{clip} \left(s_{l,t} a_g^{\text{activity}} \frac{\tau_g - \gamma_g}{2}, -\ell_{\text{max}}, \ell_{\text{max}} \right) \quad (22)$$

where $s_{l,t}$ is the scheduled gauge strength and $\ell_{\max}$ is the maximum absolute log-scale move. It then rescales the matrices as

$$A_{l,g} \leftarrow e^{\ell_g} A_{l,g}, \quad B_{l,g} \leftarrow e^{-\ell_g} B_{l,g}. \quad (23)$$

In the homogeneous limit, equation 23 changes only the representative. With the RMS floor, exact function preservation is lost near the floor; the applied log move remains bounded by $|\ell_g| \leq \ell_{\max}$.

Algorithm 1: MatrixPolicy step. Given gradients $\nabla \mathcal{L}(\theta^t)$ and base learning rate η_t :

1. Update EMAs of RLB group pressure and rational-coefficient activity.
2. Apply the centered, clipped group multipliers in equation 9 to A_l and B_l gradients.
3. Step non-matrix parameters, including rational coefficients, with the AdamW-backed non-matrix optimizer.
4. Step each RLB matrix with equation 18.
5. Apply the bounded positive-gauge rebalance in equation 23.

4 Experiments

5 Discussion

6 Conclusion

References

- Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Hieu Pham, Xuanyi Dong, Thang Lu, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. Symbolic Discovery of Optimization Algorithms. In *Advances in Neural Information Processing Systems*, 2023. URL https://papers.neurips.cc/paper_files/paper/2023/hash/9a39b4925e35cf447ccba8757137d84f-Abstract-Conference.html.
- Aaron Defazio, Xingyu Yang, Ahmed Khaled, Konstantin Mishchenko, Harsh Mehta, and Ashok Cutkosky. The Road Less Scheduled. In *Advances in Neural Information Processing Systems*, 2024. URL <https://neurips.cc/virtual/2024/poster/96925>.
- Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations*, 2015. URL <https://mlanthology.org/iclr/2015/kingma2015iclr-adam/>.
- Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Gadre, Hritik Bansal, Etash Guha, et al. DataComp-LM: In Search of the Next Generation of Training Sets for Language Models. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024. doi: 10.52202/079017-0455. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/19e4ea30dded58259665db375885e412-Abstract-Datasets_and_Benchmarks_Track.html.
- Kaizhao Liang, Lizhang Chen, Bo Liu, and Qiang Liu. Cautious Optimizers: Improving Training with One Line of Code. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=zBPZeRjfgu>.
- Hong Liu, Zhiyuan Li, David Hall, Percy Liang, and Tengyu Ma. Sophia: A Scalable Stochastic Second-order Optimizer for Language Model Pre-training. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/06960915ba8674c7a898ec0b472b80ff-Abstract-Conference.html.

- Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In International Conference on Learning Representations, 2019. URL <https://openreview.net/forum?id=Bkg6RiCqY7>.
- Yang Luo, Xiaozhe Ren, Zangwei Zheng, Zhuo Jiang, Xin Jiang, and Yang You. CAME: Confidence-guided Adaptive Memory Efficient Optimization. In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics, pp. 4442–4453, 2023. doi: 10.18653/v1/2023.acl-long.243. URL <https://aclanthology.org/2023.acl-long.243/>.
- Matteo Pagliardini, Pierre Ablin, and David Grangier. The AdEMAMix Optimizer: Better, Faster, Older. In International Conference on Learning Representations, 2025. URL <https://iclr.cc/virtual/2025/poster/28625>.
- Guilherme Penedo, Hynek Kydlicek, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro von Werra, and Thomas Wolf. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. In Advances in Neural Information Processing Systems, Datasets and Benchmarks Track, 2024. doi: 10.52202/079017-0970. URL https://papers.nips.cc/paper/2024/hash/370df50ccfd8bde18f8f9c2d9151bda-Abstract-Datasets_and_Benchmarks_Track.html.
- Luca Soldaini, Rodney Kinney, Akshita Bhagia, Dustin Schwenk, David Atkinson, Russell Authur, Ben Bogin, Khyathi Chandu, et al. Dolma: An Open Corpus of Three Trillion Tokens for Language Model Pretraining Research. arXiv preprint arXiv:2402.00159, 2024. URL <https://arxiv.org/abs/2402.00159>.
- Nikhil Vyas, Depen Morwani, Rosie Zhao, Itai Shapira, David Brandfonbrener, Lucas Janson, and Sham Kakade. SOAP: Improving and Stabilizing Shampoo using Adam for Language Modeling. In International Conference on Learning Representations, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/e988664070e9591f93fdcf605f7dc623-Abstract-Conference.html.
- Kaiyue Wen, David Leo Wright Hall, Tengyu Ma, and Percy Liang. Fantastic Pretraining Optimizers and Where to Find Them. In International Conference on Learning Representations, 2026. URL <https://openreview.net/forum?id=2J51qUZ0iG>.
- Yushun Zhang, Congliang Chen, Ziniu Li, Tian Ding, Chenwei Wu, Diederik P. Kingma, Yinyu Ye, Zhi-Quan Luo, and Ruoyu Sun. Adam-mini: Use Fewer Learning Rates To Gain More. In International Conference on Learning Representations, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/45ae878717399e6f62d57c65f052cd46-Abstract-Conference.html.
- Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuan-dong Tian. GaLore: Memory-Efficient LLM Training by Gradient Low-Rank Projection. In International Conference on Machine Learning, 2024. URL <https://openreview.net/forum?id=hYHsrKDiX7>.

A Design Rationale and Mathematical Details

This appendix separates exact facts from design heuristics. The exact facts are that the main denominator has no real poles and that RLB has an exact positive gauge in the homogeneous-radius limit. The design heuristics are the use of derivative/output gains as cheap function-space proxies, the use of relative gradient pressures as on-policy imbalance signals, and the use of a short early matrix-normalized branch for RLB matrices. We do not claim a convergence theorem, a natural-gradient derivation, or exact gauge preservation under the stabilized RMS floor.

A.1 Pole-Safe Global Rational Curves

For each group, the global-rational activation uses the denominator

$$Q(u) = 1 + |b_1||u| + |b_2|u^2 + |b_3||u|^3 + |b_4|u^4. \quad (24)$$

Every term is nonnegative for real u , hence $Q(u) \geq 1$. This is the main reason for this denominator form: it preserves rational expressivity without introducing real poles. The price is that the denominator is only piecewise smooth because of the absolute-value terms. This is acceptable for the intended use: the curve is initialized from a SiLU fit, trained with standard automatic differentiation, and used as a pole-safe trainable scalar response inside each RLB group.

Using no local atoms has both a priori and empirical motivation. A priori, the global-rational activation has a smaller and cleaner optimizer surface: numerator and denominator parameters only, no atom coefficients, no atom statistics, and no empty atom parameter group. Empirically, our ablations use the same global-rational activation for both MatrixPolicy and non-MatrixPolicy RLB controls, so the main comparison is not confounded by a MatrixPolicy-only local atom path.

A.2 Positive Gauge Lemma

Consider one group and set $\epsilon_{\text{rms}} = 0$. For $r(z) = \sqrt{m^{-1}\|z\|_2^2}$, any $a > 0$, and any nonzero group with $r(z) > 0$,

$$r(az) = ar(z), \quad u(az) = \frac{az}{r(az)} = \frac{z}{r(z)} = u(z). \quad (25)$$

Therefore

$$h(az) = r(az)R(u(az)) = ar(z)R(u(z)) = ah(z). \quad (26)$$

If the corresponding output columns are scaled by a^{-1} , their contribution to $B_l h_l$ is unchanged. Applying the argument independently to all groups proves equation 6.

With the stabilizing floor, the normalized coordinate after scaling becomes

$$u_\epsilon(az) = \frac{az}{\sqrt{a^2 m^{-1}\|z\|_2^2 + \epsilon_{\text{rms}}}} = \kappa(a, z)u_\epsilon(z), \quad (27)$$

where

$$\kappa(a, z) = \frac{a\sqrt{m^{-1}\|z\|_2^2 + \epsilon_{\text{rms}}}}{\sqrt{a^2 m^{-1}\|z\|_2^2 + \epsilon_{\text{rms}}}}. \quad (28)$$

When $m^{-1}\|z\|_2^2 \gg \epsilon_{\text{rms}}$, $\kappa(a, z) \approx 1$; when a group is near the floor, the normalized input changes and exact invariance is lost. This is why gauge rebalance is exact only in the homogeneous idealization. With the stabilized RMS floor, only the applied log-scale move is explicitly bounded by clipping.

A.3 Why Derivative and Output Gains Are Reasonable Proxies

A perturbation of the output matrix has first-order effect

$$\delta y_g = \delta B_{l,g} h_{l,g}. \quad (29)$$

MatrixPolicy uses $o_g = \text{rms}(R_g(u_{l,g}))$, the RMS of the scalar rational output on normalized coordinates, as a cheap output-side proxy. This abstracts away the group radius $r_{l,g}$ and is not the full B -update Jacobian. A perturbation of $A_{l,g}$ first perturbs $z_{l,g}$ and then passes through the normalized rational branch. Ignoring the surrounding Transformer context and keeping only the dominant scalar nonlinearity term gives a sensitivity proportional to the rational derivative; hence $d_g = \text{rms}(R'_{l,g}(u_{l,g}))$ is a cheap input-side proxy. The full Jacobian also contains the derivative of the RMS normalization and interactions with the rest of the network, so this is not a Fisher, Hessian, or natural-gradient claim.

The centered inverse-gain rule in equation 9 follows from this proxy: large-gain groups receive smaller raw matrix-gradient scale and small-gain groups receive larger scale. Before the final clip,

$$\frac{1}{G} \sum_{g=1}^G \log(c_g / \text{geomean}(c)) = 0, \quad (30)$$

so the operation is a redistribution of update budget in log scale. The final clip bounds the influence of noisy per-group statistics and can slightly shift the mean log multiplier.

A.4 Why Pressure Appears in Both Scaling and Gauge Balance

In the homogeneous gauge direction, the infinitesimal scale transform is $A_g \mapsto e^s A_g$, $B_g \mapsto e^{-s} B_g$. If the loss were exactly invariant and no regularization or optimizer state were present, the directional derivative would satisfy

$$\langle \nabla_{A_g} \mathcal{L}, A_g \rangle_F - \langle \nabla_{B_g} \mathcal{L}, B_g \rangle_F = 0. \quad (31)$$

MatrixPolicy does not use this inner product directly; instead it uses the more robust relative-pressure proxy

$$\pi_g = \log q_{\text{in},g} - \log q_{\text{out},g}. \quad (32)$$

Positive π_g means the input matrix is receiving larger relative gradient pressure than the output matrix. MatrixPolicy uses this signal to reduce input matrix scale and increase output matrix scale in the group-gradient multiplier, and to shift the gauge target toward a representative with less persistent relative-update imbalance. This is an a priori heuristic grounded in the RLB gauge, not an exact quotient-space optimizer.

The gauge target

$$\tau_g = w_\tau \frac{1}{2} (\log d_g - \log o_g) + \beta_p \bar{\pi}_g \quad (33)$$

combines the local sensitivity proxy with the clipped on-policy pressure proxy, where $\bar{\pi}_g$ is clipped using the gauge-balance threshold $p_{\text{max}}^{\text{bal}}$. The applied move is scaled by schedule and activity and clipped to $[-\ell_{\text{max}}, \ell_{\text{max}}]$, so the update is a bounded move toward τ_g , not exact canonicalization.

A.5 Why the Matrix-Normalized Branch Is Early

The early branch in equation 18 is restricted to RLB matrices and to the beginning of training. The a priori reason is that the matrices A_l and B_l define domains and recombinations for the rational groups; early matrix conditioning and rotations can matter before the rational curves and surrounding Transformer settle. The a posteriori reason is that generic Muon controls do not explain the final MatrixPolicy advantage. The branch is therefore a short matrix-family component of MatrixPolicy, after which the update decays to the role/depth AdamW matrix policy.

A.6 A Posteriori Design Constraints

The empirical design constraints are intentionally narrower than the full design space. The main method is the global-rational RLB sublayer in Section 3.1 combined with the MatrixPolicy optimizer in Section 3.3. Local rational atoms are an ablation of the activation family, not part of the main method. Likewise, the empirical claim is not that the activation alone explains the result: matched RLB controls with the same activation but different optimizers are required to isolate the optimizer contribution.

This also fixes the accounting boundary for experiments. Failed or unstable optimizer pairings should be reported under the same stopping rules as stable runs, and runtime comparisons should use clean training time measured by the training process. Non-training wall-clock delay and discarded interrupted attempts are systems artifacts rather than properties of an optimizer or activation.

B MatrixPolicy Hyperparameter Details

For $L > 1$, normalized depth is $d_l = (l - 1)/(L - 1)$. MatrixPolicy uses

$$\rho_{\text{in}}(l) = \text{clip}(1 - 0.50(d_l - 0.5), 0.55, 1.40), \quad (34)$$

$$\rho_{\text{out}}(l) = \text{clip}(1 + 1.00(d_l - 0.5), 0.55, 1.40), \quad (35)$$

$$a_{\text{role}}(l, r) = \max(0.10, 1 + 1.20(\rho_r(l) - 1)). \quad (36)$$

The AdamW matrix multiplier is

$$a_{l,r} = \text{clip}(3.0a_{\text{role}}(l, r), 0.40, 4.0). \quad (37)$$

Let $\xi_t = t/T$ and let $\sigma_{a,b}(\xi)$ be a smoothstep gate from 0 to 1 over $[a, b]$. The early matrix-normalized branch uses

$$\mu_{l,r,t} = \text{clip}\left(0.75\sigma_{0.02,0.12}(\xi_t)[1 - \sigma_{0.20,0.36}(\xi_t)]\rho_r(l)s_{\text{stat}}(l, r, t), 0, 0.75\right), \quad (38)$$

where $s_{\text{stat}}(l, r, t)$ is the bounded pressure/activity penalty

$$s_{\text{stat}}(l, r, t) = \text{clip}\{\exp[-0.30P_l(t) - 0.65A_l(t)], 0.10, 1.15\}, \quad (39)$$

$$P_l(t) = G^{-1} \sum_{g=1}^G |\text{clip}(\pi_g, -1.50, 1.50)|, \quad (40)$$

$$A_l(t) = G^{-1} \sum_{g=1}^G \text{ReLU}\left(\frac{\alpha_g - 0.05}{0.45}\right). \quad (41)$$

In the main configuration, $\mu_{l,r,t} = 0$ after the decay window.

The gauge rebalance runs every five optimizer steps. Its schedule is

$$s_{l,t} = 0.50\sigma_{0.03,0.35}(\xi_t) \text{clip}(1 + 0.15(d_l - 0.5), 0.75, 1.25), \quad (42)$$

with $\ell_{\text{max}} = 0.030$. The pressure contribution in the gauge target uses EMA decay 0.95, pressure coefficient $\beta_p = 0.25$, and pressure clip $p_{\text{max}}^{\text{bal}} = 1.25$ for the gauge calculation. Rational activity scales the applied move with $\beta_a = 0.10$ and clamp range $(a_{\text{min}}, a_{\text{max}}) = (0.75, 1.25)$.

The main MatrixPolicy configuration enables the group-gradient policy with

$$\lambda_g = 0.20, \quad \lambda_p = 0.10, \quad \lambda_a = 0.20, \quad (c_{\text{min}}, c_{\text{max}}) = (0.75, 1.35), \quad (43)$$

and a smooth activation window from training progress 0.02 to 0.30.

The constants are deliberately modest. The exponent values keep group redistribution in the small-perturbation regime; the clips prevent a single noisy group from dominating the matrix update; the smooth windows suppress statistics from the cold-start phase and remove the matrix-normalized branch before the long-run optimizer comparison. In the experiments, these constants are fixed across datasets and model sizes rather than tuned per run.